What Is a Robot?

And why robot software needs a framework

Prof. Anis Koubaa Dr. Asem Ibrahim Alalwan

EE 414 — Introduction to Robotics
College of Engineering
Alfaisal University

Fall 2026 — Week 1, Lecture B

What this lecture does

Roadmap

Four questions: what a robot is, what one is made of, what makes the physical world hard, and why nobody writes robot software as a single program any more.

1. What is a robot?
2. Anatomy and the autonomy pipeline
3. Three hard truths
4. Why robot software needs a framework

Outline

Part 1

What is a robot?

Which of these is a robot?

- A washing machine
- A CNC milling machine
- A robot vacuum cleaner
- An elevator
- A large language model
- A self-driving car
- An automatic sliding door
- A factory welding arm

Decide before the next slide. Say why.

The disagreements you have right now are the whole content of the next ten minutes.

A working definition

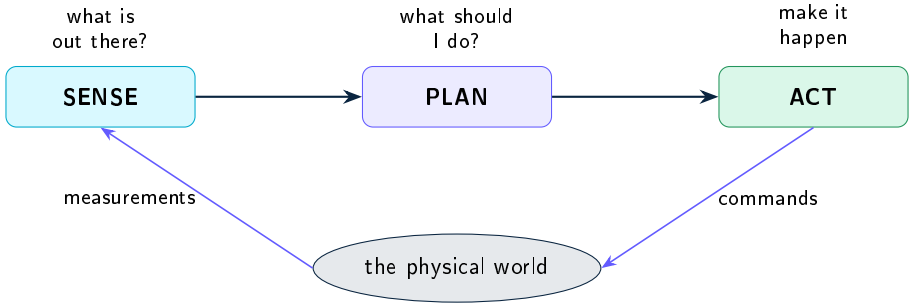
A robot **senses** its environment, **decides** what to do,
and **acts** on the physical world.

All three are required. Drop any one and you have something else:

Missing	What you have	Example
Sensing	An automaton — it repeats, it does not react	Washing machine, CNC
Deciding	A teleoperated tool — a human is the loop	Surgical master–slave arm
Acting	A perception or reasoning system	A vision model, an LLM

The boundary is genuinely fuzzy, and arguing about it is not useful. What is useful is knowing *which of the three parts* a given machine is missing.

The loop that never stops



Why it is drawn as a loop and not a line

Acting changes the world, which changes what you sense, which changes what you decide. A robot never finishes a pass — it runs this loop tens of times per second until you switch it off.

Automation is not autonomy

	Automation	**Autonomy**
Question it answers	"Repeat this exactly."	"Achieve this goal, whatever happens."
The environment	Known and controlled — built around the machine	Unknown and changing — the machine must cope
Failure mode	Something is out of place; it does the wrong thing anyway	Something is out of place; it must notice and adapt
Example	Welding arm on a fixed jig	Warehouse robot in a corridor with people in it

This course is about the right-hand column. That is why estimation, mapping and planning take six of our eleven weeks — they are the machinery of coping.

Robots, sorted by where they work

Family	Typical form	The hard part
Industrial arms	Fixed base, 6 joints, high precision	Speed and accuracy; safety near people
Mobile robots	Wheeled base indoors — AMRs, service, cleaning	Knowing where it is; not hitting anything
Aerial (UAV)	Multirotor or fixed wing	Unstable by nature; no room for error
Field robots	Agriculture, mining, inspection, marine	No map, no GPS, no help nearby
Humanoids and legged	Legs, arms, whole-body control	Balance, contact, and everything at once

This course lives in row 2 — the wheeled mobile robot. It is the shortest honest path to a complete autonomous system, and every idea in it transfers upward.

Why mobile robots, and why now

- **Warehousing and logistics** — the largest deployed fleet of autonomous mobile robots in the world, and the reason Nav2 is engineered the way it is.
- **Inspection** — pipelines, plants, substations. Sending a robot where the environment is hostile, repetitive, or simply large.
- **Service and cleaning** — hospitals, airports, malls. Shared space with people, which makes safety the binding constraint rather than speed.
- **Agriculture** — monitoring and treatment at plant-level resolution.

In the field

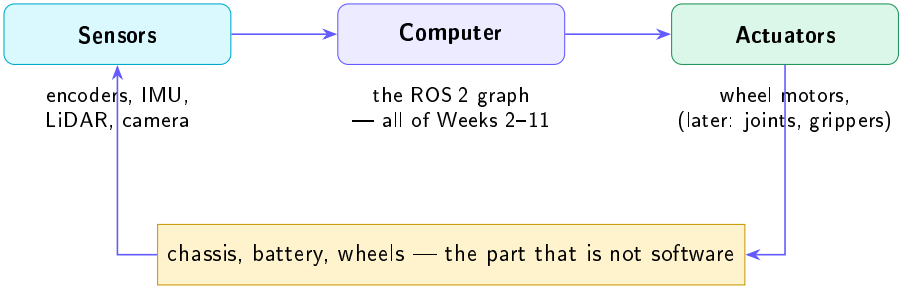
In the region, industrial inspection and logistics are where robotics hiring is concentrated, and Vision 2030 names industrial automation explicitly. The stack in this course is the one those roles ask for. That is not a coincidence — it is why the course was rebuilt.

Outline

Part 2

Anatomy and the autonomy pipeline

What a robot is made of



Everything you will write in this course lives in the middle box. The other two decide what is *possible*, and they are the reason the middle box is difficult.

The sensors, and what each one lies about

Sensor	What it gives you	How it lets you down
Wheel encoders	How far each wheel turned	Wheels slip. The error *accumulates* and never recovers.
IMU	Angular rate, acceleration	Drifts. Integrate it twice and the answer is fiction.
LiDAR	Distance to obstacles, all around	Blind to glass; sparse far away; 10 Hz, not continuous.
Camera	Dense colour and texture	Needs light; depth is inferred, not measured.
GNSS / GPS	Absolute position outdoors	Useless indoors; metres of error where it works.

The idea that fixes this

No single sensor is trustworthy, but their failures are *different*. Combining them so the errors cancel is sensor fusion — Week 9, and why the Kalman filter is here.

You already know how this works

A wheelchair is a differential drive.

What the hands do	What the chair does
Push both rims equally	Straight line
Push one, hold the other	Turns on the spot
Push one harder than the other	Curves — an arc

Two wheels, driven separately, and no steering mechanism anywhere. Every motion the chair can make comes out of the *difference* between the two — which is where the name comes from.

Now: how would you tell a computer to do that?

The actuator you will spend the semester commanding

The differential drive: two independently driven wheels, one passive caster.

- Both wheels same speed $\rightarrow$ straight line
- Opposite speeds $\rightarrow$ turn in place
- Anything else $\rightarrow$ an arc

You will not command wheel speeds. You will command

$$v \text{ (m/s)}, \qquad \omega \text{ (rad/s)}$$

and something else will work out the wheels.

Week 4 in one line

That "something else" is the differential-drive kinematic model. Deriving it, and knowing when it stops being true, is Week 4.

In the field

(v, ω) is a ROS 2 message called `geometry_msgs/Twist`, published on a topic called `/cmd_vel`. Nearly every wheeled robot on earth accepts it. Learn it once, drive anything.

You are the robot. Go to the door.

A corridor. The door is nine metres away. Someone has left a bag on the floor that is not on any map. You have a LiDAR and two wheel encoders, and nothing else.

What do you have to work out, and in what order?

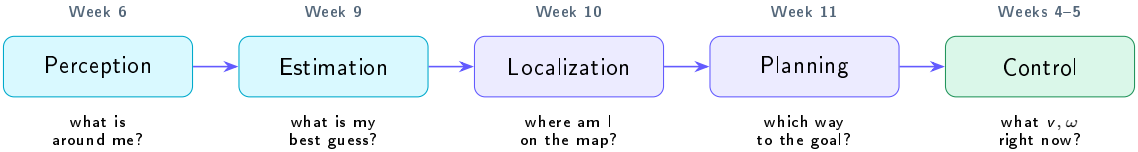
Shout them at me. Every answer you give is one week of this course.

Five questions, and none of them can be skipped

The question	What answers it
"The LiDAR just gave me 360 distances. Is that a map of anything?"	**Perception**
"Something solid is 2.1 m ahead of me. Fine — but where am *I*?"	**Estimation**
"I have a guess about my own motion. Where does it put me *on the map*?"	**Localization**
"I am here, the door is there, the bag is between us. Which way?"	**Planning**
"The path says go that way. What do the two wheels do in the next 100 ms?"	**Control**

Each one exists because the one before it did not answer the question.

The same five, and they are the course



Five boxes. Every autonomous mobile robot ever built has these five, whatever the vendor calls them. **The rest of this course is one box at a time**, and Week 12 is the week they all run together.

"Go to the door", with real numbers in it

Perception	The LiDAR returns 360 distances. Most are walls. A cluster at 2.1 m ahead is not on the map — something is there.
Estimation	Encoders say I moved 0.42 m; the IMU says I also turned 3° that I never commanded. Fused, the best estimate lies between.
Localization	That estimate, matched against the map, puts me at $(4.2, 1.8)$ facing north — with a confidence, not a certainty.
Planning	From $(4.2, 1.8)$ to the door at $(9.0, 1.5)$: a path exists, and it goes around the unmapped cluster.
Control	The next 100 ms of that path means a forward speed of 0.22 m/s and a turn rate of -0.15 rad/s. Publish it.

Then do the whole thing again. Ten times a second.

Outline

Part 3

Three hard truths

Do this now, with your eyes shut

Stand up. Close your eyes. Walk ten steps forward, turn around, walk ten steps back.

You are not where you started.

You knew how many steps you took. You knew you turned around. You were still wrong by half a metre — because you were **counting your own motion** instead of looking at the room.

This is exactly what a wheel encoder does

It counts rotations. If a wheel slips 2 % of the time — carpet, dust, a cable — then after 50 m of driving the robot is **a metre** from where it believes it is. And nothing in that calculation ever tells it so. The error only grows.

Truth 1 — every measurement is wrong

A robot never knows where it is. It has a **belief**, and a measure of how much to trust it.

What a beginner writes

`x = 4.2`

"The robot is at 4.2 metres."

What robotics writes

`x = 4.2` $\pm$ `0.3`

"Most likely 4.2, and I would not bet the robot on the third digit."

Under the hood

This is why Weeks 9–10 are probabilistic. A planner that trusts a wrong position drives confidently into a wall — the confidence is the bug, not the position.

Truth 2 — nothing is instantaneous

Between the world changing and the wheels responding, there is a chain of delays:

Add them up: about 175 ms. At the 0.22 m/s this course drives at, the robot travels **4 cm after it has already decided to stop** — and the thing it saw has moved too.

You are always steering by a slightly old picture of the world.

In the field

Four centimetres sounds survivable. Multiply the speed by ten, as a warehouse AMR does, and it is 40 cm. This is why a controller that is perfect in a simulator with no delay oscillates on real hardware, and why we tune in simulation *with* realistic rates.

Truth 3 — everything runs at once

These are not steps in a program. They are all running, always, at different rates:

Job	Must run at	Because
Read the LiDAR	10 Hz	that is how fast it spins
Read wheel encoders	50 Hz	odometry error grows if you miss samples
Run the controller	20 Hz	smooth motion
Re-plan the path	1 Hz	planning is expensive
Update the map	5 Hz	the world changes

So: how do you write *one program* that does all five, at five different rates?

You do not. That is the whole point of the next section.

Outline

Part 4

Why robot software needs a framework

The obvious way to write it

```
while True:
    scan  = read_lidar()          # blocks 100 ms
    odom  = read_encoders()
    pose  = estimate(odom, scan)
    path  = plan(pose, goal)      # 200 ms, and not needed every cycle
    v, w  = control(path, pose)
    send_to_motors(v, w)
```

This is correct, readable, and it is what everyone writes first. It is also unusable, for five separate reasons.

Put a stopwatch on it

`read_lidar()` — one full sweep	100 ms
`plan()` — searching the map	200 ms
everything else	20 ms
one trip round the loop	**320 ms**

So the controller runs at **3 Hz**. You designed it for 20 Hz.

At 0.22 m/s that is 7 cm of travel between decisions — half the robot's length, with its eyes shut.

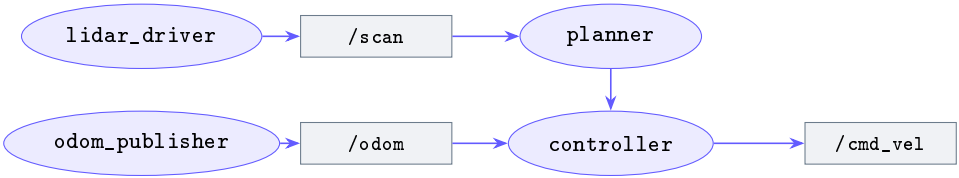
And planning did not need to run this cycle at all. Nor the one before it.

Five reasons that loop fails

1. **One rate for everything.** The whole loop runs at the speed of its slowest line — the 320 ms you just measured. Your 20 Hz controller runs at 3 Hz.
2. **One crash kills everything.** A malformed camera frame takes down the motor commands, and the robot keeps rolling with the last velocity it was given.
3. **One machine.** You cannot move the heavy vision work to a second computer without rewriting the program.
4. **Nothing is reusable.** Change the LiDAR model and you edit the same file that holds your controller.
5. **You cannot see inside it.** When the robot turns the wrong way, there is nothing to inspect — no way to ask "what did the planner actually output?"

Every one of these is a *structural* problem. No amount of careful coding inside that loop fixes them.

The answer: many small programs and a bus



Each box is a **separate operating-system process** with its own rate and its own crash. They exchange named, typed messages. Nothing calls anything directly — `lidar_driver` does not know the planner exists.

This is a **robotics middleware**. The one we use is ROS 2.

Each of the five problems, answered

The problem	What the graph does about it
One rate for everything	Each node has its own timer. The planner being slow does not slow the controller.
One crash kills everything	A node dies alone. The others keep running, and it can be restarted.
One machine	Nodes find each other over the network. Moving one to another computer changes no code.
Nothing is reusable	Swap the LiDAR driver for a different model. Everything downstream still gets `/scan`.
You cannot see inside it	Every message is inspectable live, and recordable to disk for replay.

ROS 2, honestly

What it is

- A way for many programs to exchange typed messages
- A build system and package format
- Command-line tools to inspect a running robot
- A large ecosystem of solved problems

What it is *not*

- An operating system. The name is historical.
- Intelligence. It moves your data; it does not decide anything.
- A guarantee of anything. A badly designed graph is still badly designed.

Under the hood

ROS 2 is a rewrite, not an update. ROS 1 had a single central master process — if it died, the robot went deaf. ROS 2 has no master: nodes discover each other directly. That change is why it is used in products and ROS 1 largely was not.

What the ecosystem hands you

Package	Week	What you would otherwise write yourself
`tf2`	7	All coordinate-frame bookkeeping, with time
`robot_localization`	9	A tuned EKF for sensor fusion
`slam_toolbox`	10	A production SLAM system
`Nav2`	11	Planners, controllers, costmaps, recovery behaviours
Gazebo, RViz2	8	A physics simulator and a live 3D inspector

In the field

Each is years of work by large teams. **Using them is the job** — the skill is choosing, configuring, tuning and debugging them, and knowing when the default is wrong for your robot. That is what Weeks 9–11 assess.

A first look at a node

A ROS 2 node that drives the robot forward. You will write this in Week 4 — read it now for the *shape*, not the detail.

```python
class DriveForward(Node):
    def __init__(self):
        super().__init__('drive_forward')
        self.pub = self.create_publisher(Twist, '/cmd_vel', 10)
        self.create_timer(0.1, self.tick)      # 10 Hz, forever

    def tick(self):
        msg = Twist()
        msg.linear.x = 0.2                     # 0.2 m/s forward
        self.pub.publish(msg)
```

No `while` loop. You declare *what happens on each tick*, and the framework calls you. Every node in this course has this shape.

And what you can see from outside it

While that node runs, in another terminal:

```
$ ros2 topic list
/cmd_vel
/odom
/scan

$ ros2 topic echo /cmd_vel
linear:
  x: 0.2
  y: 0.0
angular:
  z: 0.0
```

You did not add logging to make this work. **Every message on every topic is inspectable, always** — and this is the single most useful debugging property the framework has.

What we established today

1. A robot **senses**, **decides** and **acts** — all three, in a loop that never stops.
2. Autonomy is not automation. Coping with an environment you did not control is the entire difficulty.
3. Five blocks: perception, estimation, localization, planning, control. The rest of this course is one block at a time.
4. Three hard truths: measurements are wrong, nothing is instantaneous, everything runs at once.
5. Therefore robot software is **many small programs exchanging messages**, not one loop — and that is what ROS 2 gives us.

Next week: what that message-passing actually is, and your first node.

Before Week 2

1. Install ROS 2 Jazzy — see `setup/` in the course repository.
   Budget 60–90 minutes. Do not leave it to the night before.
2. Run the setup check script. Every row must say `PASS`. Bring the output.
3. Set your assigned `ROS_DOMAIN_ID`.
4. Read the ROS 2 "Concepts" page. Skim, do not study — the vocabulary is what matters.

One thought to bring back

Pick any machine you used today. Which of sense, decide, act does it have? Which does it lack? That question is the whole of today's lecture, and it is worth ten seconds a day for the rest of the semester.